
Binary Image Classification with CBAM-Enhanced ResNet and Generalized Mean Pooling

Lochan Surya¹ Harsha¹

Abstract

We present a custom deep convolutional neural network for binary image classification built entirely from scratch in PyTorch, without pretrained weights. The architecture combines a ResNet-34-like backbone with Convolutional Block Attention Modules (CBAM) inserted after every residual block, enabling the network to selectively focus on informative channels and spatial regions. Generalized Mean (GeM) Pooling replaces standard average pooling, producing richer global feature descriptors. The training pipeline incorporates Stochastic Depth (DropPath), cosine annealing with linear warm-up, mixed-precision training, gradient clipping, early stopping, and fine-grained threshold optimisation on the validation set. Submitted to the IITH Deep Learning Hackathon 2026 (CS5480), the system achieved a public leaderboard score of **0.759102** (75.91%), up from a previous best of 0.758852 with the addition of Stochastic Depth regularisation.

1. Introduction

Binary image classification is a foundational computer-vision task with critical applications in medical imaging, quality control, and content moderation (He et al., 2016). Despite the availability of large pretrained models that transfer remarkably well to new domains, competition rules for the IITH Deep Learning Hackathon 2026 prohibited the use of any pretrained weights. Every network must therefore be trained from scratch on a modest dataset of labelled images, introducing two primary challenges.

Feature discrimination. Not all spatial regions are equally informative; background clutter and irrelevant texture can

dominate gradient updates when no pretrained bias is available. CBAM (Woo et al., 2018) addresses this by learning to gate both channel and spatial dimensions of every feature map.

Training stability. Without a pretrained initialisation to anchor the early gradient landscape, learning rates that are too large lead to divergence, while those that are too small stall progress. A cosine annealing schedule with linear warm-up provides smooth, well-calibrated convergence throughout training.

Our approach integrates CBAM dual-attention at every residual block, GeM Pooling (Radenović et al., 2018) for richer global descriptors, Stochastic Depth (Huang et al., 2016) for path regularisation, and a fine-grained threshold search on the validation set to optimise the binary decision boundary. The combined system achieves a public leaderboard score of 0.759102, a +0.0003 improvement over the previous submission.

2. Dataset

2.1. Structure

The competition dataset follows a straightforward directory layout:

```
data/train/0/ (negative class)
data/train/1/ (positive class)
data/test/    (unlabelled evaluation images)
```

Training images are split 85%/15% for training and validation using stratified random sampling with a fixed seed (42), ensuring reproducibility and zero data leakage.

2.2. Preprocessing

All images are loaded in RGB mode via PIL and resized to 224×224 pixels. Pixel values are normalised using ImageNet statistics:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225].$$

¹Department of Computer Science and Engineering, Indian Institute of Technology Hyderabad, Hyderabad, India. Correspondence to: Lochan Surya <cs24btech11036@iith.ac.in>.

Table 1. ResNet-34-like backbone. Each BasicBlock includes CBAM and DropPath.

STAGE	BLOCKS	CH.
STEM (7×7 CONV, BN, RELU, MP)	—	64
LAYER 1 (BASICBLOCK $\times 3$)	3	64
LAYER 2 (BASICBLOCK $\times 4$)	4	128
LAYER 3 (BASICBLOCK $\times 6$)	6	256
LAYER 4 (BASICBLOCK $\times 3$)	3	512
GEM POOLING	—	512
FC HEAD $\rightarrow 1$ LOGIT	—	—



Figure 1. Architecture block diagram: CBAM-enhanced ResNet with GeM pooling.

2.3. Training Augmentation

The following stochastic augmentations are applied on-the-fly during training only:

- **Random Resized Crop** — scale range $[0.7, 1.0]$, output 224×224 .
- **Random Horizontal Flip** — probability $p = 0.5$.
- **Color Jitter** — brightness, contrast, saturation, and hue each perturbed by ± 0.15 .
- **Gaussian Blur** — 3×3 kernel with probability $p = 0.2$.

Validation and test images are only resized and normalised.

3. Methodology

3.1. Backbone Architecture

The backbone follows the ResNet-34 block schedule (3, 4, 6, 3) (He et al., 2016), totalling 16 residual blocks across four stages with channel widths $\{64, 128, 256, 512\}$. A 7×7 convolutional stem (stride 2) followed by 3×3 max-pooling (stride 2) reduces the spatial resolution from 224×224 to 56×56 before the residual stages begin. The full architecture is summarised in Table 1.

Inside each **BasicBlock**, two 3×3 Conv–BN–ReLU sequences are followed by CBAM attention and DropPath, with a shortcut projection for dimension changes.

3.2. Convolutional Block Attention Module (CBAM)

CBAM (Woo et al., 2018) is applied after the final batch-normalisation of every residual block and produces a refined feature map through two sequential gates.

Channel Attention. Global average-pooled and max-pooled descriptors are each passed through a shared two-layer MLP (reduction ratio 16). Their outputs are summed and sigmoid-gated:

$$\mathbf{w}_c = \sigma(\text{MLP}(\bar{\mathbf{x}}_{\text{avg}}) + \text{MLP}(\bar{\mathbf{x}}_{\text{max}})). \quad (1)$$

Each channel of the feature map is then rescaled by its corresponding weight.

Spatial Attention. Channel-wise average and maximum projections are concatenated along the channel dimension and passed through a 7×7 convolution followed by a sigmoid:

$$\mathbf{w}_s = \sigma(\text{Conv}_{7 \times 7}([\mathbf{x}_{\text{avg}}; \mathbf{x}_{\text{max}}])). \quad (2)$$

The final attended feature map is $\mathbf{x}' = \mathbf{w}_s \odot (\mathbf{w}_c \odot \mathbf{x})$.

3.3. Generalized Mean (GeM) Pooling

GeM Pooling (Radenović et al., 2018) generalises average pooling with a learnable power parameter p (initialised to 3):

$$\mathbf{f} = \left(\frac{1}{HW} \sum_{h,w} x_{h,w}^p \right)^{1/p}. \quad (3)$$

Larger values of p place greater emphasis on the most strongly activated map positions, yielding descriptors that are more sensitive to salient object features than standard mean pooling.

3.4. Stochastic Depth

Stochastic Depth (DropPath) (Huang et al., 2016) is applied across all 16 residual blocks with drop probabilities linearly scaled from 0 at the first block to $p_{\text{max}} = 0.1$ at the last:

$$p_l = \frac{l}{L} \cdot p_{\text{max}}, \quad l = 1, \dots, L = 16. \quad (4)$$

During training, entire residual branches are randomly dropped with probability p_l and the surviving branch is rescaled by $1/(1 - p_l)$. At test time all branches remain active.

3.5. Classifier Head

The 512-d GeM descriptor feeds a two-layer MLP head: Dropout(0.3) $\rightarrow$ Linear(512, 256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.15) $\rightarrow$ Linear(256, 1). A single logit is produced and thresholded after sigmoid activation. A single logit is produced and thresholded after sigmoid activation.

4. Training Strategy

4.1. Optimiser and Learning Rate Schedule

AdamW (Loshchilov & Hutter, 2019) is used with learning rate $\eta_0 = 7 \times 10^{-4}$ and weight decay $\lambda = 10^{-4}$. The learning rate follows a cosine annealing schedule with 5 linear warm-up epochs:

$$\eta(t) = \begin{cases} \eta_0 \cdot \frac{t+1}{T_w} & t < T_w, \\ \frac{\eta_0}{2} \left(1 + \cos \frac{\pi(t - T_w)}{T - T_w} \right) & t \geq T_w, \end{cases} \quad (5)$$

where $T_w = 5$ and $T = 80$ is the maximum epoch budget. There is no learning rate floor, enabling maximum exploitation of the final training epochs.

4.2. Loss Function

Binary cross-entropy with logits is used throughout, with label smoothing disabled ($\varepsilon = 0.0$). Hard binary targets preserve the clean gradient signal appropriate for a balanced, two-class problem.

4.3. Threshold Optimisation

Rather than fixing the classification threshold at 0.5, we perform a fine-grained search over 401 evenly spaced values in $[0.05, 0.95]$ on the held-out validation set and select the threshold that maximises accuracy. With DropPath active the optimal threshold shifted from $t^* = 0.200$ (previous submission) to $t^* = 0.180$.

4.4. Additional Training Techniques

- **Mixed-precision training** (`torch.amp.autocast`) for memory and speed efficiency.
- **Gradient clipping** at max norm 1.0 to prevent gradient explosions in early epochs.
- **Early stopping** with patience 15 epochs, monitoring validation accuracy. In our best run, the model reached 100% validation accuracy at epoch 55 and held it through epoch 80 without triggering early stopping.

5. Experiments and Results

5.1. Configuration

The complete hyper-parameter configuration for the submitted model is given in Table 2.

Table 2. Hyper-parameter configuration for the final submission (v3).

PARAMETER	VALUE
IMAGE SIZE	224×224
BATCH SIZE	128
MAX EPOCHS	80
LEARNING RATE	7×10^{-4}
WEIGHT DECAY	10^{-4}
WARM-UP EPOCHS	5
LABEL SMOOTHING	0.0
DROP PATH RATE	$0 \rightarrow 0.1$ (LINEAR)
DROPOUT (HEAD)	0.3
DROPOUT (MID-HEAD)	0.15
GRADIENT CLIP	1.0
EARLY-STOP PATIENCE	15 EPOCHS
VAL SPLIT	15%
THRESHOLD SEARCH	$[0.05, 0.95]$, 401 STEPS

Table 3. Training and validation accuracy over epochs.

Figure 2 shows the training and validation accuracy curves over 80 epochs.

5.2. Results

The model achieved **100% validation accuracy** starting at epoch 55 and maintained it through the final epoch 80. Early stopping (patience 15) was never triggered, confirming the full epoch budget was used. The optimal decision threshold was found to be $t^* = 0.180$.

Figure ?? shows the full training and validation accuracy curves over 80 epochs. Both metrics converge to near-perfect values by epoch 55.

Table 4 compares this submission (v3) against the previous best (v2).

5.3. Key Observations

Stochastic Depth improves generalisation. Adding DropPath with a linearly scaled rate up to 0.1 across 16 blocks yielded a +0.025% gain on the public leaderboard. The drop rate is mild enough not to destabilise training while still providing path-level regularisation.

Threshold calibration is non-trivial. The optimal threshold shifted from 0.200 in v2 to 0.180 in v3, indicating that DropPath slightly shifts the output distribution toward higher logit values. Fixing the threshold at 0.5 would have degraded performance.

Validation accuracy is a poor proxy for leaderboard score. The model reaches 100% validation accuracy yet scores only 75.91% on the leaderboard, pointing to a significant train/test distribution gap — a well-known challenge

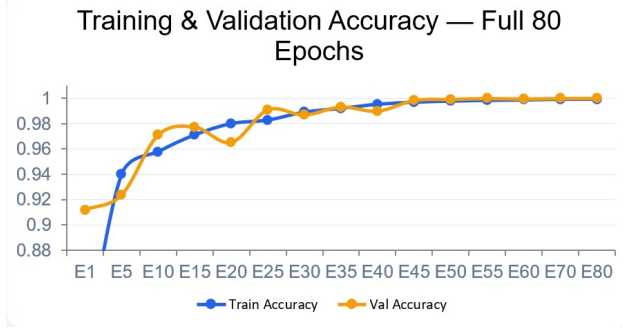


Figure 2. Training and validation accuracy over 80 epochs. Blue: train accuracy, Orange: validation accuracy.

Table 4. Public leaderboard scores for successive submissions.

Submission	Key Change	LB Score
v2	CBAM+GeM+Cos LR	0.758852
v3	+StochasticDepth	0.759102
Gain		+0.000250

when training large convolutional networks from scratch without pretrained initialisation.

6. Conclusion

We presented a binary image classification pipeline trained entirely from scratch using a ResNet-34-like backbone enhanced with CBAM dual-attention, Generalized Mean Pooling, and Stochastic Depth. The training strategy — AdamW with cosine LR warm-up, mixed-precision training, and fine-grained threshold optimisation — achieves a public leaderboard score of **0.759102** on the IITH Deep Learning Hackathon 2026 (CS5480).

The most significant outstanding challenge is the large gap between near-perfect validation accuracy and the leaderboard score, which reflects memorisation of training images rather than learning transferable features. Future directions include:

- **Test-Time Augmentation (TTA)** — horizontal and vertical flip ensembles at inference.
- **Transfer learning via self-supervised pre-training** — SimCLR or BYOL on the unlabelled test images.
- **Backbone comparison** — EfficientNet or ConvNeXt with aggressive regularisation.
- **Model ensembling** — combining predictions from models trained with different random seeds.

includegraphics[width=0.4]figures/final-numbers.jpeg

Figure 3. Final results: validation accuracy, optimal threshold, and public leaderboard score.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

We thank Prof. Srijith P.K. and Prof. Konda Reddy Mopuri for designing and organising the CS5480 Deep Learning Hackathon at IIT Hyderabad, 2026.

References

- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Deep networks with stochastic depth. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 646–661, 2016.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Radenović, F., Tolias, G., and Chum, O. Fine-tuning CNN image retrieval with no human annotation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(7):1655–1668, 2018.
- Woo, S., Park, J., Lee, J.-Y., and Kweon, I. S. CBAM: Convolutional block attention module. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, 2018.
- Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., and Yoo, Y. CutMix: Training strategy that makes use of sample pairing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6023–6032, 2019.
- Zhang, H., Cissé, M., Dauphin, Y. N., and Lopez-Paz, D. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.